

Long Short-Term Memory (1997): 긴 지연을 건너는 기억

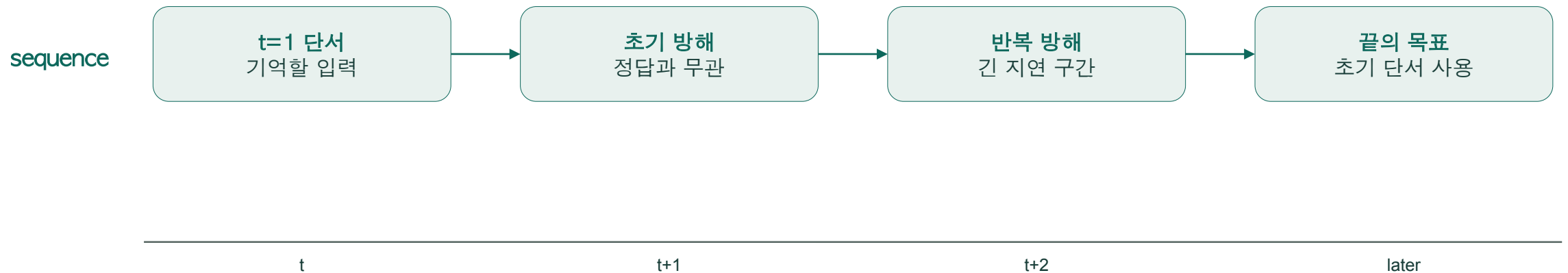
Long Short-Term Memory

Sepp Hochreiter and Jürgen Schmidhuber (1997)

원 논문의 gradient 문제, CEC, 두 gate, truncated learning rule을 함께 읽는다

긴 지연 문제의 구조

초기의 유용한 신호와 끝의 teacher signal 사이에 많은 방해 입력이 놓인다



학습은 끝에서 생긴 error를 먼 과거의 저장 결정까지 돌려보내야 한다

Recurrent gradient는 반복 곱으로 변한다

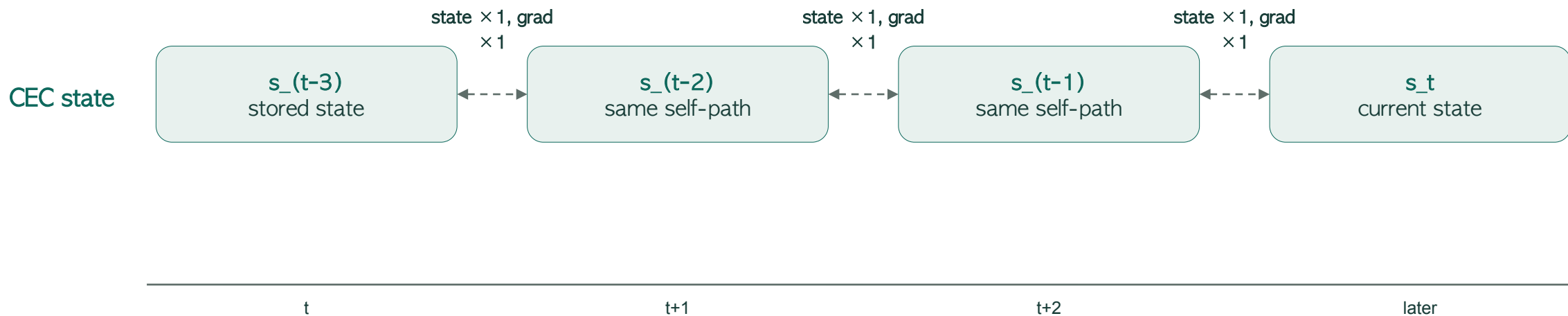
교육용으로 매 단계의 local factor가 0.9라고 두면 거리 q 가 늘수록 신호가 급감한다

거리 q	반복 곱 0.9^q	남는 크기	의미
10	0.9^{10}	0.3487	이미 약해짐
100	0.9^{100}	0.0000266	학습 신호 희박
1000	0.9^{1000}	1.75×10^{-46}	사실상 소실

각 단계의 factor가 1보다 작으면 long-range error가 거리의 지수로 감소한다

CEC는 상태와 error에 factor 1을 제공한다

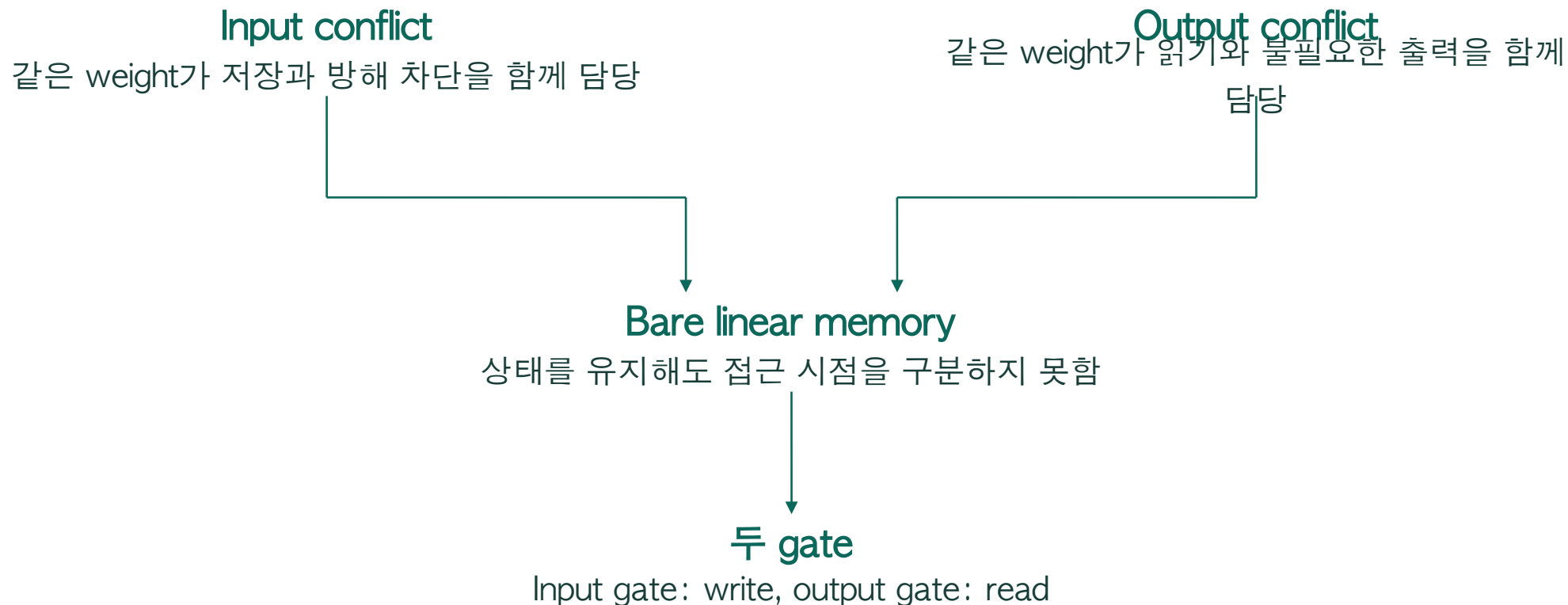
CEC는 linear unit의 self-connection weight를 1.0으로 고정한 상태 경로다



CEC 내부 derivative가 약 1이라는 주장은 원 논문의 truncation rule에 조건부다

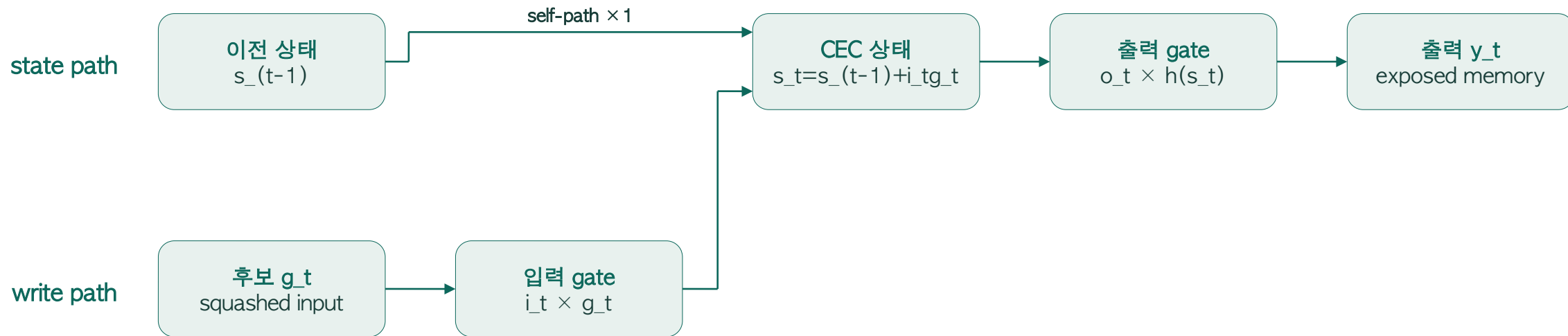
Bare CEC에는 쓰기과 읽기 충돌이 남는다

고정 self-path만으로는 어떤 입력을 저장하고 언제 출력을 숨길지 결정할 수 없다



1997년 원 논문의 memory cell

original algorithm: truncated gradient, input gate와 output gate만 사용



1997년 셀에는 forget gate와 peephole connection이 없다

원 논문 cell의 두 계산

i_t 는 input gate 출력, o_t 는 output gate 출력, g_t 는 후보, s_t 는 내부 상태다

계산

읽을 수 있는 식

gate의 역할

State update

$$s_t = s_{(t-1)} + i_t \times g_t$$

쓸 후보의 양을 선택

Cell output

$$y_t = o_t \times h(s_t)$$

외부에 보일 양을 선택

Input closed

$$i_t \approx 0$$

후보가 state를 바꾸지 못함

Output closed

$$o_t \approx 0$$

state는 남고 출력은 숨김

쓰기와 읽기는 분리되지만 과거 state 자체를 줄이는 계수는 없다

세 단계 state 계산

교육용 값이며 $s_0=0$, $h(s)=\tanh(s)$ 로 둔다

t	i_t	g_t	s_t	o_t × tanh(s_t)
1	1.0	0.60	0.60	0.00
2	0.0	-0.90	0.60	0.00
3	0.0	0.20	0.60	$0.8 \times \tanh(0.6) \approx 0.430$

t=2의 방해 후보는 input gate가 닫혀 있어 저장값 0.60을 덮어쓰지 못한다

거리와 gradient scaling의 차이

CEC의 내부 구간과 cell 경계에서 생기는 scaling을 구분한다

경로	한 단계 factor	1000단계 효과	조건
Ordinary path 예시	0.9	1.75×10^{-46}	매 단계 반복
CEC internal path	약 1	약 1	truncated rule
Cell 입구와 출구	gate와 h' , g'	경계에서 한 번씩	포화 시 작아질 수 있음

CEC는 lag 길이에 따른 반복 감쇠를 제거하지만 경계의 작은 derivative까지 없애지는 않는다

Truncated gradient가 유지하고 끊는 경로

원 논문의 constant-flow 주장은 ordinary full BPTT가 아니라 architecture-specific truncation에 의존한다

경로	원 논문의 처리	학습상 의미
이전 internal state	CEC 안에서 유지	먼 state까지 error 전달
Cell과 gate 밖의 recurrent path	더 이전으로 전파 중단	추가 순환 곱 차단
현재 incoming weight	현재 error로 update	cut이 weight 고정을 뜻하지 않음
Output connection	ordinary backprop 사용	현재 output error 반영

Error는 CEC 안에서 과거로 흐르고, cell 밖으로 나가려는 recurrent derivative는 잘린다

원 논문의 learning update 흐름

Forward state와 local derivative를 순차적으로 갱신해 sequence length와 무관한 저장량을 유지한다

01	Forward	Gate, candidate, internal state를 계산
↓		
02	Local sensitivity	필요한 state-to-weight derivative만 유지
↓		
03	Output error	현재 teacher error를 cell output에 전달
↓		
04	CEC backflow	Internal state를 따라 과거 error를 유지
↓		
05	Update and cut	Incoming weights를 갱신하고 외부 recurrence는 truncate

원 algorithm은 forward mechanism과 gradient truncation을 함께 정의한다

$O(1)$ 은 전체 학습 시간이 아니다

W 는 weight 수이며 원 논문의 local-in-space-and-time 정의를 단위별로 읽어야 한다

Claim	정확한 단위	의미
$O(1)$	weight 하나, time step 하나당	network 전체 비용 아님
$O(W)$	time step 하나의 전체 update	fully recurrent BPTT와 같은 차수
$O(W)$ storage	유지할 derivative 총량	sequence length에 독립
Full BPTT	time-unrolled activations 저장	저장량이 sequence length에 의존

효율성 주장은 per-step update와 저장량에 관한 것이며 training examples 수와는 별개다

Memory cell block과 network topology

여러 cell이 input gate와 output gate를 공유하면 distributed representation을 저장할 수 있다

- 크기 S 의 block은 S 개 memory cell과 하나의 input gate, 하나의 output gate를 공유한다.
- Hidden layer에는 memory cells와 gate units가 함께 놓이며 conventional hidden units도 추가할 수 있다.
- 논문의 실험은 task에 따라 block 수와 block 크기를 다르게 정한다.
- Output gate가 항상 필수는 아니지만 canonical Figure 1 cell에는 두 gate가 모두 있다.

LSTM은 단일 scalar cell만이 아니라 gate를 공유하는 cell block 구조로 실험됐다

1997년 원형에 포함되지 않은 요소

후대 구현에서 익숙한 기능을 원 논문의 mechanism으로 되돌려 넣지 않는다

요소	1997년 원 논문	정확한 해석
Forget gate	없음	과거 state 계수는 fixed 1
Peephole	없음	직접 state-to-gate path를 정의하지 않음
Input gate	있음	새 candidate의 write 양을 제어
Output gate	canonical cell에 있음	Stored state의 exposure를 제어
Learning	truncated RTRL variant	ordinary full BPTT와 구분

CEC를 modern forget gate가 1로 고정된 경우라고 부르면 역사와 학습 규칙이 흐려진다

Original LSTM과 later standard LSTM

두 식의 차이는 과거 state를 보존하는 coefficient가 fixed인지 learned인지에 있다

구분	State recurrence	과거 state 처리	대표 학습
Original 1997	$s_t = s_{(t-1)} + i_{tg}_t$	fixed coefficient 1	truncated rule
Later common form	$c_t = f_{tc}_{(t-1)} + i_{tz}_t$	learned forget gate f_t	commonly full BPTT

Later forget gate는 기억을 의도적으로 줄이거나 reset할 수 있는 별도 제어다

1997년 실험을 읽는 기준

Reported success는 각 synthetic task의 architecture, stopping criterion, trial 수에 묶여 있다

- 모든 주요 결과는 artificial long-lag task에서 측정됐다.
- Task별로 architecture, learning rate, output encoding과 success threshold가 다르다.
- Training sequences until success는 wall-clock speed나 sample efficiency의 보편적 척도가 아니다.
- Baseline row도 같은 조건인지 확인한 뒤 paper-specific comparison으로 읽어야 한다.

숫자는 task 조건과 trial 또는 test count를 함께 제시할 때만 의미가 유지된다

Task 2a: delay 100의 무잡음 sequence

Table 2는 18 trials의 mean과 각 method의 성공 trial 비율을 보고한다

Method	Delay	Successful trials	Success after
LSTM	100	100%	5,040 sequences
Chunker	100	33%	32,400 sequences
BPTT	100	0%	>5,000,000
RTRL	100	0%	>5,000,000

이 task와 stopping rule에서는 LSTM이 delay 100을 안정적으로 학습했다

Task 2c: 천 단계 이상의 최소 지연

p는 distractor symbol 수, q+1은 minimum lag이며 각 (p,q) 조건을 20 trials로 평가했다

p	q	Minimum lag	Weights	Success after
1000	1000	1001	6064	49,000 sequences
500	1000	1001	3064	49,000 sequences
200	1000	1001	1264	75,000 sequences

LSTM은 짧은 training sequence 없이도 noisy distractor 사이의 장기 dependency를 학습했다

Continuous adding problem

두 marked real values의 합을 sequence 끝에서 예측하며 $T/2$ 가 minimum lag다

T	Minimum lag	Test errors	Success after	Trials
100	50	1 / 2560	74,000	mean of 10
500	250	0 / 2560	209,000	mean of 10
1000	500	1 / 2560	853,000	mean of 10

원 논문은 binary latch뿐 아니라 continuous value 저장과 계산도 시험했다

Temporal order problem

Widely separated X와 Y의 순서를 기억해 sequence 끝에서 class를 고른다

Task	Relevant symbols	Test errors	Success after	Trials
6a	2	1 / 2560	31,390	mean of 20
6b	3	2 / 2560	571,100	mean of 10

여러 장기 단서를 순서대로 저장하는 난이도는 6b에서 training sequences 증가로 나타났다

실험이 지지하는 범위

원 논문의 결과와 그 결과만으로 결론낼 수 없는 내용을 분리한다

Supported by 1997 experiments

특정 synthetic tasks에서 long lag 학습

Noise와 distributed representation 처리

Continuous adding과 temporal order 성공

Reported baselines보다 많은 successful runs

Not established by these experiments

Modern speech와 NLP의 우월성

모든 real-world distribution 일반화

모든 non-decomposable task 해결

현재 architecture 간 보편적 ranking

1997년 evidence는 mechanism 가능성을 보였고 real-world 성능은 future work로 남겼다

한계 1 : strongly delayed XOR

첫 input 하나만 저장해도 expected error가 줄지 않는 non-decomposable task는 truncated rule이 어렵게 만든다

01 First bit 혼자서는 target 정보가 부족



02 Long delay 즉시 개선할 local subgoal이 없음



03 Second bit 두 bit를 함께 봐야 XOR 결정



04 Truncated credit 첫 저장 결정까지 유용한 signal 전달이 어려움

CEC가 long lag를 보존해도 objective가 단계적으로 분해되지 않으면 학습은 어렵다

한계 2: state drift와 output saturation

Forget gate가 없는 additive state는 입력 편향이 누적되면 큰 절댓값으로 drift할 수 있다

문제	영향	1997년 mitigation	남는 경계
Mostly same-sign input	State magnitude 증가	Input gate를 초기에 닫히게 bias	Active forgetting 없음
Large state	$h'(s)$ 가 작아짐	h 선택과 gate bias	출구 gradient 감소 가능
Cell abuse	Bias cell처럼 점유	Negative output-gate bias	추가 allocation 필요 가능

CEC 내부 factor 1과 usable end-to-end gradient는 같은 개념이 아니다

한계 3: 정확한 discrete counting과 적용 범위

기억을 오래 유지하는 능력만으로 99단계와 100단계를 항상 정확히 구별하지는 못한다

- 정확한 elapsed-step counting에는 별도 counting mechanism이 필요할 수 있다.
- 짧은 lag와 긴 lag의 대략적 구분이나 learned decay는 가능하다.
- 논문의 tasks는 인공 데이터이며 real-world application은 future work로 제시됐다.
- 결론은 original algorithm과 1997년 evidence 범위 안에서 읽어야 한다.

Long memory, exact clock, broad real-world generalization은 서로 다른 주장이다

1997년 LSTM의 설계 논리

문제 분석과 architecture, learning rule이 하나의 연쇄를 이룬다

01	Repeated products	Long-range error가 사라지거나 폭발
↓		
02	CEC	Fixed self-connection으로 내부 factor를 1에 고정
↓		
03	Two gates	Write와 read conflict를 분리
↓		
04	Truncation	CEC 밖 recurrent gradient를 중단
↓		
05	Synthetic evidence	천 단계 이상 lag와 연속값 task를 시험

LSTM의 original contribution은 cell 모양만이 아니라 error path를 설계한 전체 방법이다

원 논문을 정확히 설명하는 문장

Mechanism, learning condition, historical boundary를 한 번에 포함한다

정확한 설명

CEC는 truncated rule 아래에서 length-independent internal error path를 만든다

Input gate는 write, output gate는 exposure를 제어한다

1997년 cell은 fixed self-path를 사용한다

Synthetic tasks에서 long lag 학습을 보였다

피해야 할 단순화

모든 gradient가 언제나 constant다

Gate가 모든 vanishing gradient를 해결한다

Original cell에 forget gate가 있다

Modern sequence task 전반의 우월성을 증명했다

CEC, 두 gate, truncation을 함께 말해야 1997년 LSTM을 정확히 설명할 수 있다
